

Software Tools and Techniques for CSE: Assessment 3

Project Report

Ridham Patel (23110238), Shardul Junagade (23110297)

108 November 2025

Computer Science & Engineering
Indian Institute of Technology Gandhinagar
India

Repository Link: `cs202-stt`

Contents

1	Lab 9: Development of C# Console Applications	2
1.1	Introduction	2
1.2	Tools	2
1.3	Setup	3
1.4	Methodology and Results	5
1.4.1	Task 1 – Environment check and basic I/O	5
1.4.2	Task 2 – Arithmetic + even/odd	6
1.4.3	Task 3 – Loops and factorial	8
1.4.4	Task 4 – Arrays and matrices	9
1.5	Output Reasoning (Tasks 5–7)	12
1.5.1	Task 5 – Level 0 (Code A)	12
1.5.2	Task 5 – Level 0 (Code B)	13
1.5.3	Task 6 – Level 1 (Code A)	13
1.5.4	Task 6 – Level 1 (Code B: Top-level statements)	14
1.5.5	Task 7 – Level 2 (Code A)	15
1.5.6	Task 7 – Level 2 (Code B)	16
1.6	Conclusion	17
1.7	References	17
2	Lab 10: Object Lifecycle and Polymorphism in C#	19
2.1	Introduction	19
2.2	Setup and Environment	19
2.3	Constructors and Data Control	19
2.3.1	Objective	19
2.3.2	Class Design	19
2.3.3	Creating and Using Objects	20
2.3.4	Observed Output	21
2.3.5	Explanation of Output	21
2.4	Inheritance and Method Overriding	21
2.4.1	Objective	21
2.4.2	Base Class: Vehicle	21
2.4.3	Derived Class: Car	22
2.4.4	Derived Class: Truck	22
2.4.5	Demonstration of Runtime Polymorphism	23
2.4.6	Observed Output	23
2.4.7	Explanation of Output	23
2.5	Output Reasoning (Level 0)	23
2.6	Output Reasoning (Level 1)	24
2.7	Output Reasoning (Level 2)	26
2.8	Output Reasoning (Level 3)	28

1 Lab 9: Development of C# Console Applications

Metadata

- Course: CS202 – Software Tools and Techniques for CSE
- Lab Topic: Development of C# Console Applications
- Name: Shardul Junagade (23110297)
- Date: 13th October 2025

Repository path: lab9/

1.1 Introduction

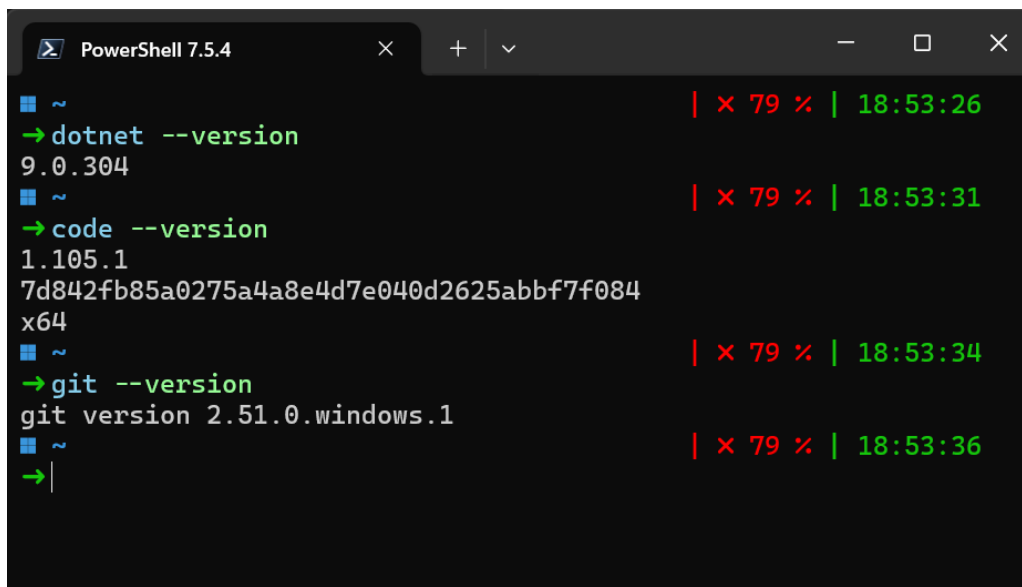
This lab was a gentle, hands-on tour of C# and .NET using small console programs. I split the work into four focused projects to keep each idea clear: a quick environment check, arithmetic with a simple class, a few loop patterns plus a factorial function, and a set of array/matrix tasks. I also reasoned through the output of the snippets in Tasks 5–7 so the “why” behind each output was explicit.

1.2 Tools

- OS: Windows 11; Shell: PowerShell 7
- .NET SDK: .NET 8 (meets the “.NET 6 or later” requirement)
- IDE/Editor: Visual Studio 2022 Community and/or VS Code
- Git: for version control (optional but handy)

Project layout used in this lab:

- lab9/src/Task1HelloWorld/
- lab9/src/Task2Arithmetic/
- lab9/src/Task3LoopsFunctions/
- lab9/src/Task4ArraysMatrices/



```
PowerShell 7.5.4
~ | x 79 % | 18:53:26
→ dotnet --version
9.0.304
~ | x 79 % | 18:53:31
→ code --version
1.105.1
7d842fb85a0275a4a8e4d7e040d2625abbf7f084
x64
~ | x 79 % | 18:53:34
→ git --version
git version 2.51.0.windows.1
~ | x 79 % | 18:53:36
→ |
```

Figure 1: Environment Details

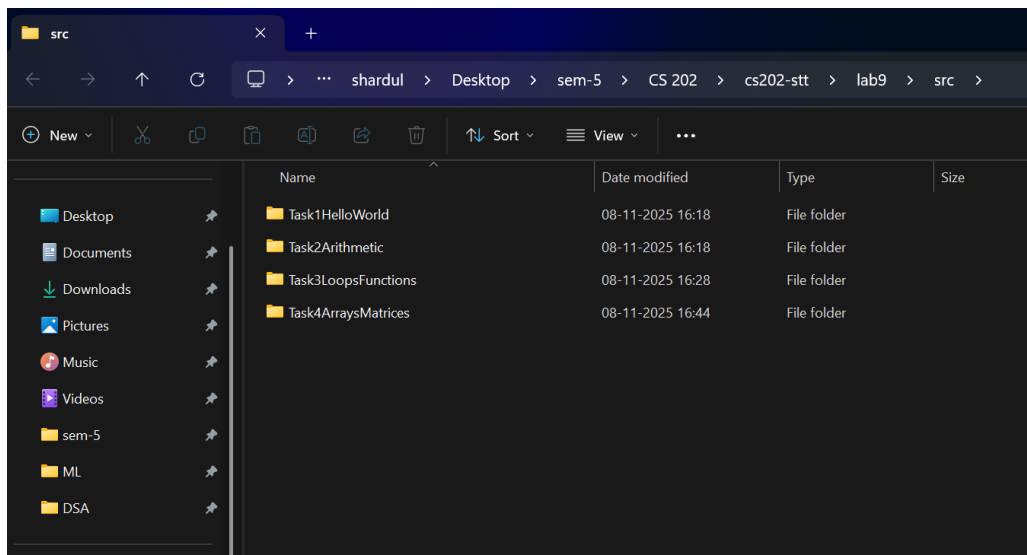


Figure 2: Project Structure

1.3 Setup

I set up a fresh .NET environment on Windows and validated it with a tiny console app before working on the lab projects.

Visual Studio path (recommended by the lab)

1. Install Visual Studio 2022 (Community Edition) with the “.NET desktop development” workload selected.

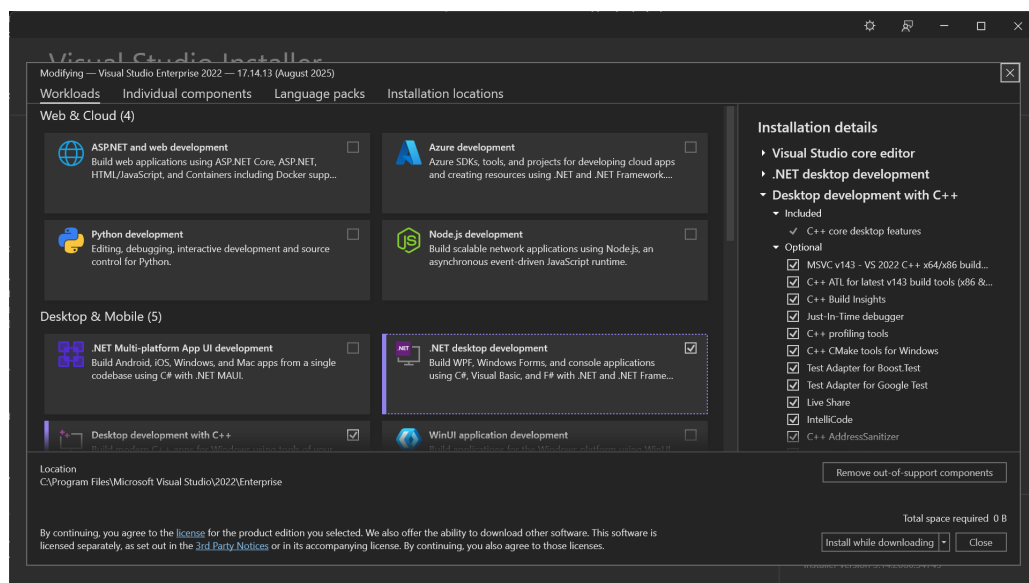


Figure 3: Workload Selection

2. Confirm the SDK is on PATH:

```
dotnet --version
```

3. Create a new C# Console App in Visual Studio and set the Target Framework to .NET 8.0 in project properties.

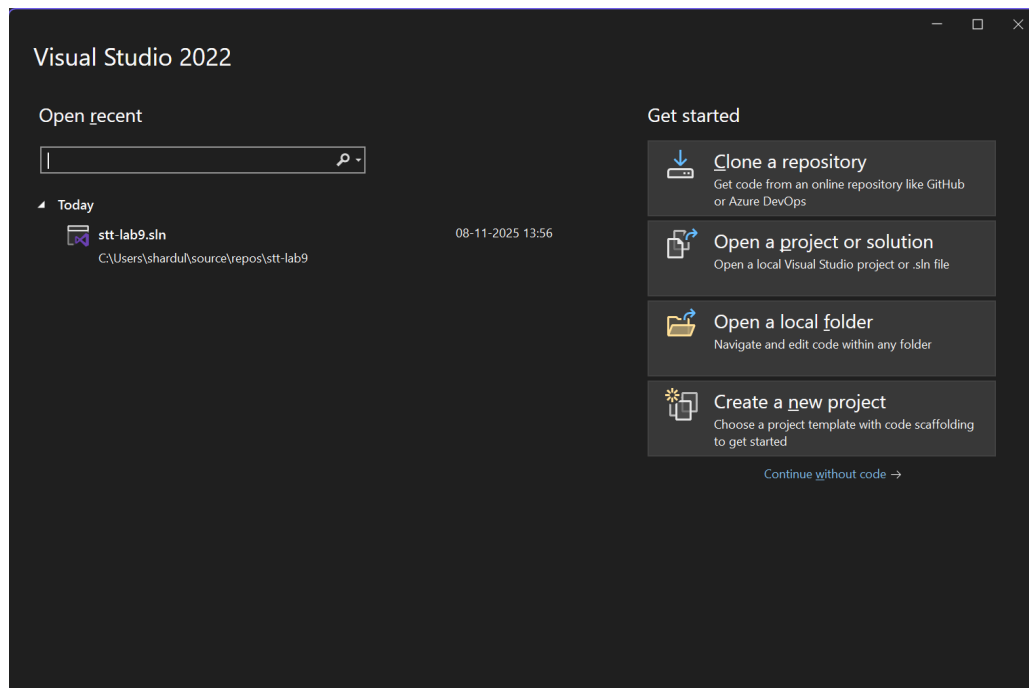


Figure 4: New Project Wizard

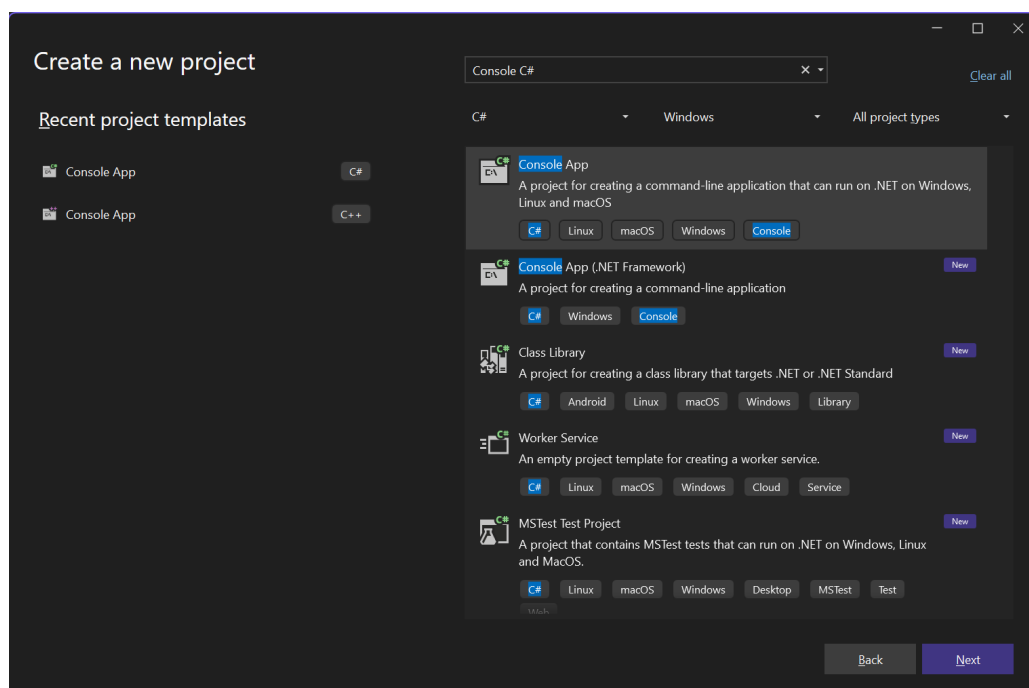


Figure 5: Project Target Framework

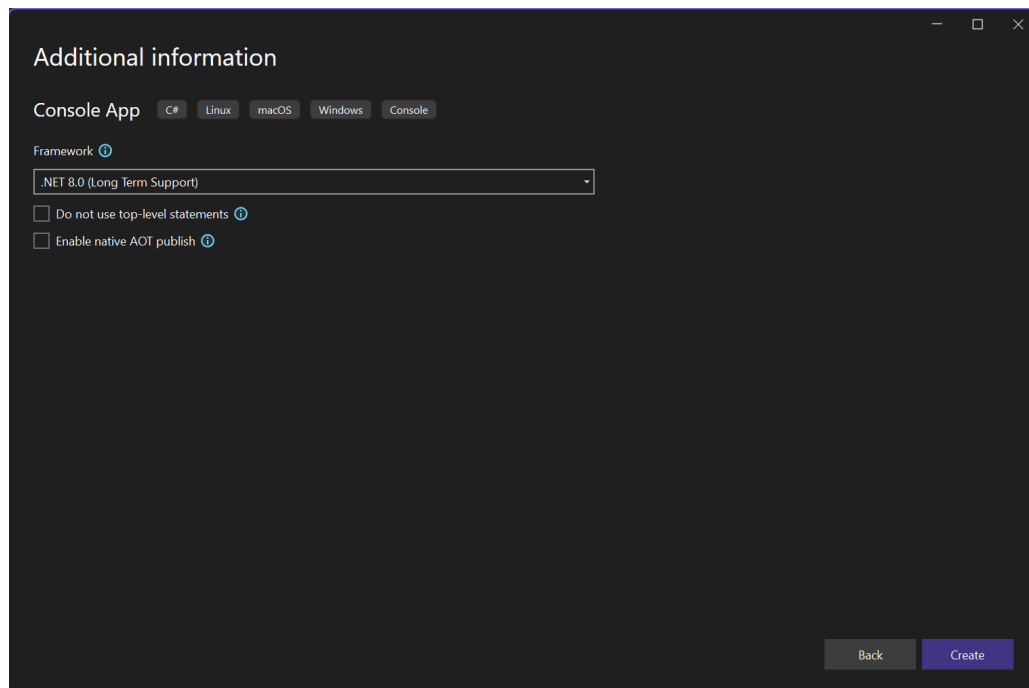


Figure 6: .NET Version Selection

CLI alternative (works well from VS Code)

```
# Create and run a throwaway test app
dotnet new console -f net8.0 -o sandbox-console
dotnet run --project sandbox-console/sandbox-console.csproj
```

1.4 Methodology and Results

1.4.1 Task 1 – Environment check and basic I/O

Code intent: confirm console I/O and runtime. Reads a name via `Console.ReadLine()` and prints a greeting.
Path: lab9/src/Task1HelloWorld/Program.cs

Run:

```
dotnet run --project lab9/src/Task1HelloWorld/Task1HelloWorld.csproj
```

Result: Prompts for a name and echoes a greeting; verifies SDK and terminal interaction.

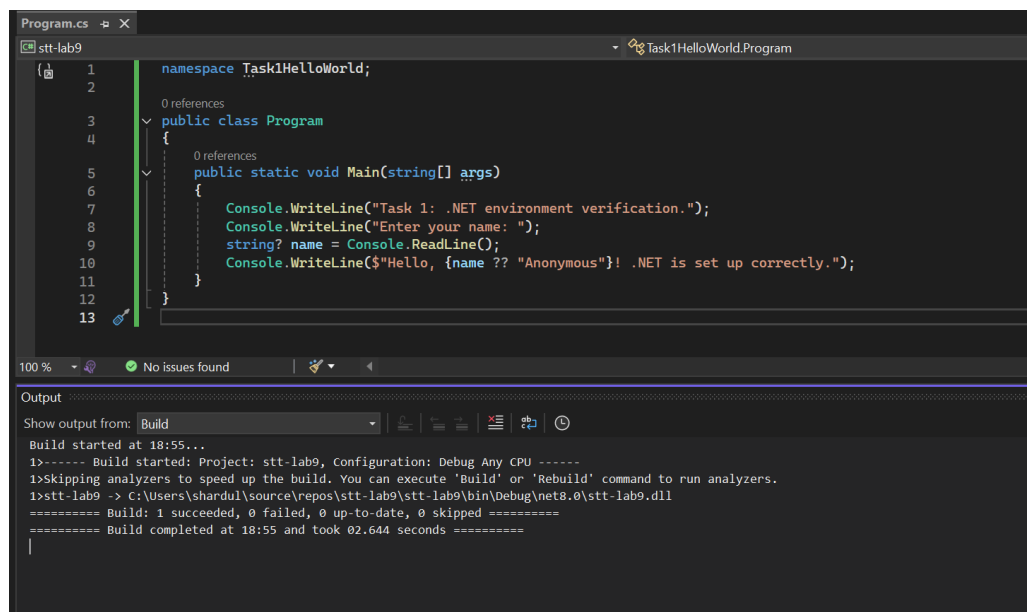
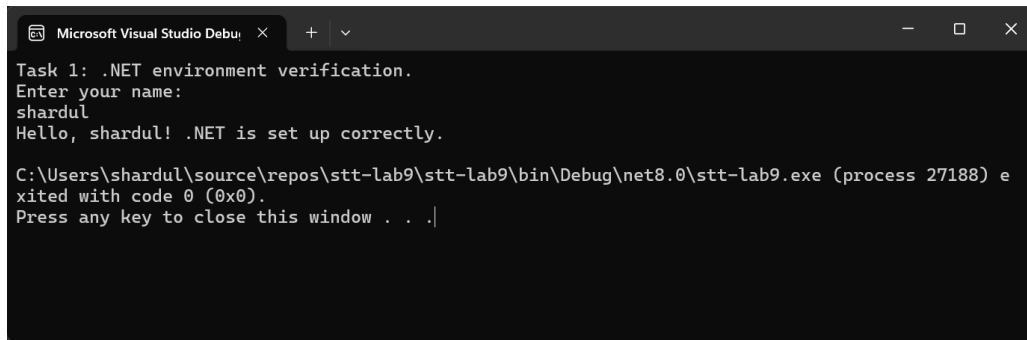


Figure 7: Task1 code snippet



```
Microsoft Visual Studio Debug Console
Task 1: .NET environment verification.
Enter your name:
shardul
Hello, shardul! .NET is set up correctly.

C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 27188) e
xited with code 0 (0x0).
Press any key to close this window . . .|
```

Figure 8: Task1 console output

1.4.2 Task 2 – Arithmetic + even/odd

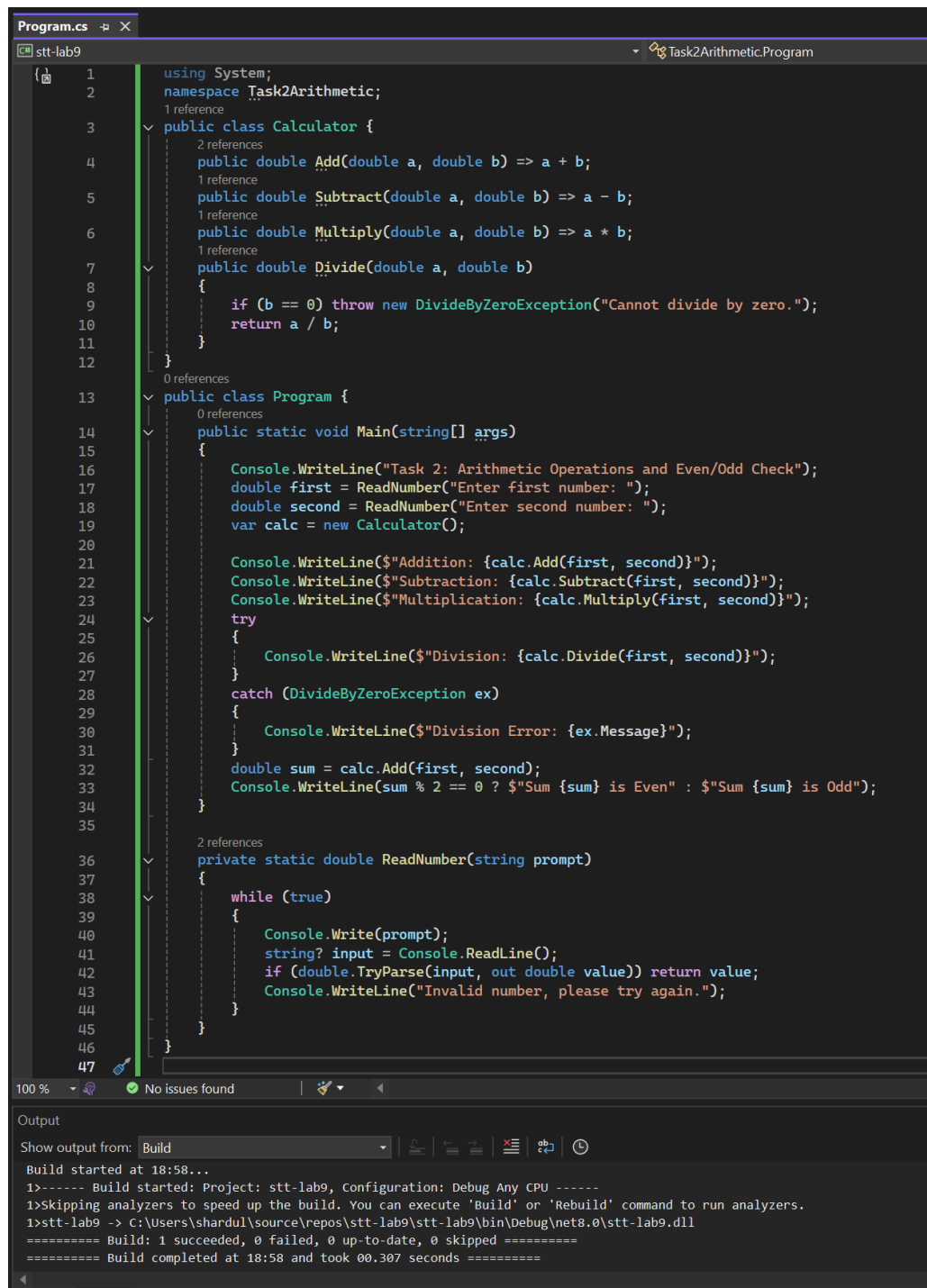
Design: Small `Calculator` class with explicit methods (`Add`, `Subtract`, `Multiply`, `Divide`). Division guarded against zero with an exception caught at call site.

Path: `lab9/src/Task2Arithmetic/Program.cs`

Run:

```
dotnet run --project lab9/src/Task2Arithmetic/Task2Arithmetic.csproj
```

Result: Prints four operation results and states whether the sum is even or odd. Division by zero produces a clear message.



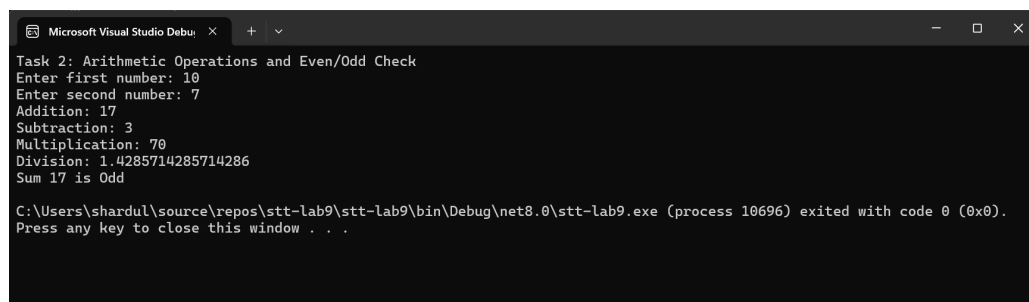
```

Program.cs
stt-lab9
Task2Arithmetic.Program

1 using System;
2 namespace Task2Arithmetic;
3 public class Calculator {
4     public double Add(double a, double b) => a + b;
5     public double Subtract(double a, double b) => a - b;
6     public double Multiply(double a, double b) => a * b;
7     public double Divide(double a, double b)
8     {
9         if (b == 0) throw new DivideByZeroException("Cannot divide by zero.");
10        return a / b;
11    }
12 }
13 public class Program {
14     public static void Main(string[] args)
15     {
16         Console.WriteLine("Task 2: Arithmetic Operations and Even/Odd Check");
17         double first = ReadNumber("Enter first number: ");
18         double second = ReadNumber("Enter second number: ");
19         var calc = new Calculator();
20
21         Console.WriteLine($"Addition: {calc.Add(first, second)}");
22         Console.WriteLine($"Subtraction: {calc.Subtract(first, second)}");
23         Console.WriteLine($"Multiplication: {calc.Multiply(first, second)}");
24         try
25         {
26             Console.WriteLine($"Division: {calc.Divide(first, second)}");
27         }
28         catch (DivideByZeroException ex)
29         {
30             Console.WriteLine($"Division Error: {ex.Message}");
31         }
32         double sum = calc.Add(first, second);
33         Console.WriteLine(sum % 2 == 0 ? $"Sum {sum} is Even" : $"Sum {sum} is Odd");
34     }
35
36     private static double ReadNumber(string prompt)
37     {
38         while (true)
39         {
40             Console.Write(prompt);
41             string? input = Console.ReadLine();
42             if (double.TryParse(input, out double value)) return value;
43             Console.WriteLine("Invalid number, please try again.");
44         }
45     }
46 }
47
100 % No issues found
Output
Show output from: Build
Build started at 18:58...
1>----- Build started: Project: stt-lab9, Configuration: Debug Any CPU -----
1>skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab9 -> C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 18:58 and took 00.307 seconds =====

```

Figure 9: Task2 code (Calculator class)



```

Microsoft Visual Studio Debug:
Task 2: Arithmetic Operations and Even/Odd Check
Enter first number: 10
Enter second number: 7
Addition: 17
Subtraction: 3
Multiplication: 70
Division: 1.4285714285714286
Sum 17 is Odd
C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 10696) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 10: Task2 sample output

1.4.3 Task 3 – Loops and factorial

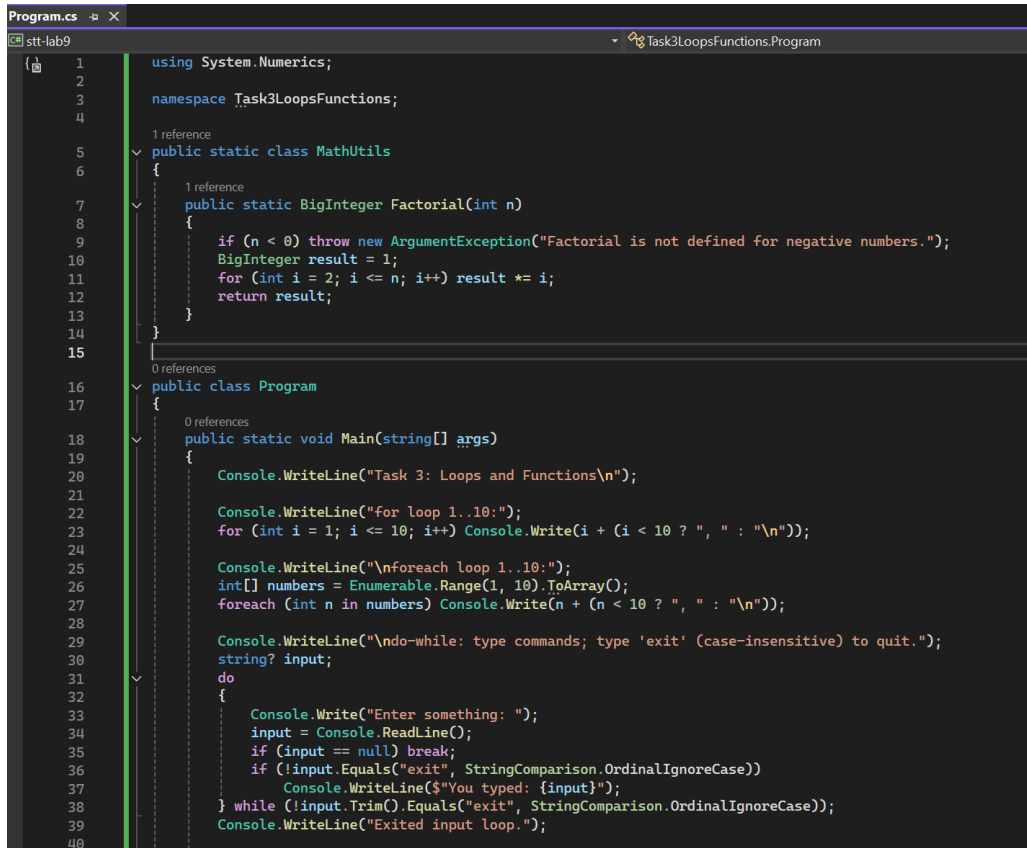
Design: Demonstrates for, foreach, and a do-while loop. Factorial provided by MathUtils.Factorial(int) returning BigInteger to avoid premature overflow.

Path: lab9/src/Task3LoopsFunctions/Program.cs

Run:

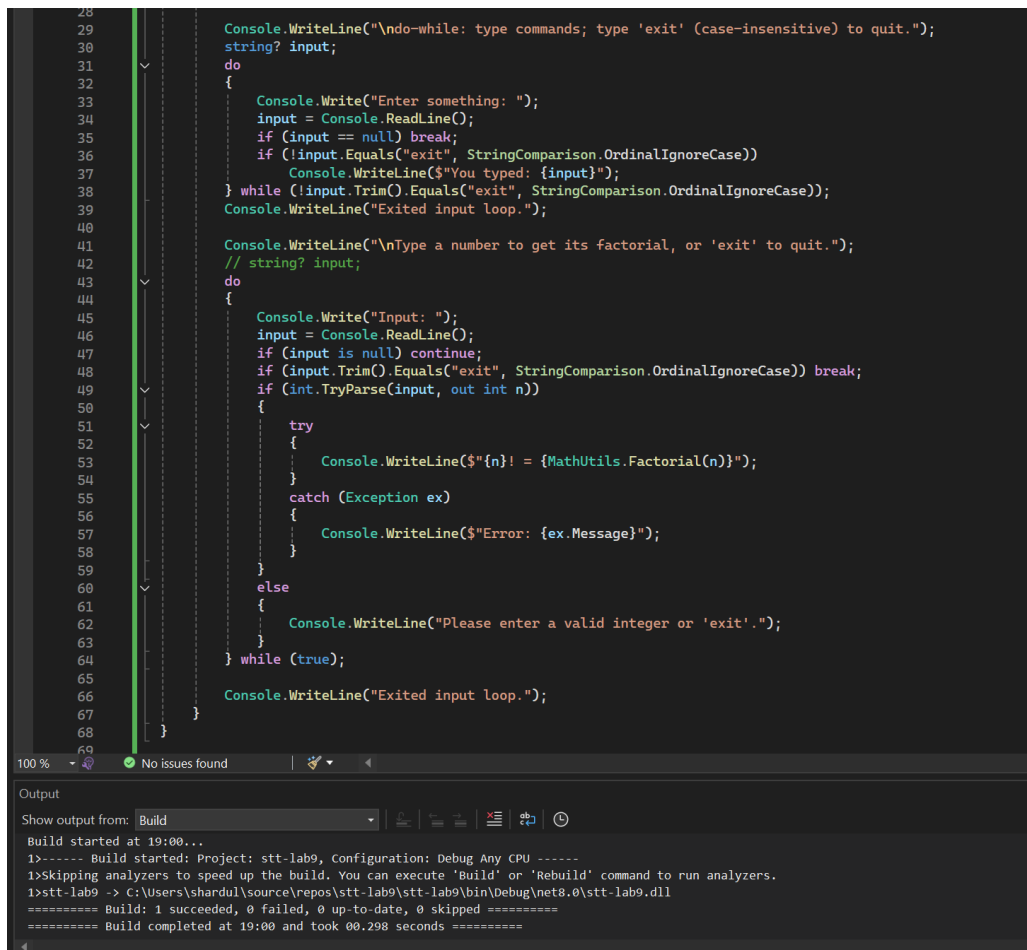
```
dotnet run --project lab9/src/Task3LoopsFunctions/Task3LoopsFunctions.csproj
```

Result: Two sequences 1..10 printed; interactive loop ends cleanly on exit; factorial results shown for valid integers, error for negatives.



```
1 using System.Numerics;
2
3 namespace Task3LoopsFunctions;
4
5 1 reference
6 public static class MathUtils
7 {
8     1 reference
9     public static BigInteger Factorial(int n)
10     {
11         if (n < 0) throw new ArgumentException("Factorial is not defined for negative numbers.");
12         BigInteger result = 1;
13         for (int i = 2; i <= n; i++) result *= i;
14         return result;
15     }
16 }
17
18 0 references
19 public class Program
20 {
21     0 references
22     public static void Main(string[] args)
23     {
24         Console.WriteLine("Task 3: Loops and Functions\n");
25
26         Console.WriteLine("for loop 1..10:");
27         for (int i = 1; i <= 10; i++) Console.Write(i + (i < 10 ? ", " : "\n"));
28
29         Console.WriteLine("\nforeach loop 1..10:");
30         int[] numbers = Enumerable.Range(1, 10).ToArray();
31         foreach (int n in numbers) Console.Write(n + (n < 10 ? ", " : "\n"));
32
33         Console.WriteLine("\ndo-while: type commands; type 'exit' (case-insensitive) to quit.");
34         string? input;
35         do
36         {
37             Console.Write("Enter something: ");
38             input = Console.ReadLine();
39             if (input == null) break;
40             if (input.Equals("exit", StringComparison.OrdinalIgnoreCase))
41                 Console.WriteLine($"You typed: {input}");
42             } while (!input.Trim().Equals("exit", StringComparison.OrdinalIgnoreCase));
43         Console.WriteLine("Exited input loop.");
44     }
45 }
```

Figure 11: Task3 code snippet



```

28
29 Console.WriteLine("\ndo-while: type commands; type 'exit' (case-insensitive) to quit.");
30 string? input;
31 do
32 {
33     Console.Write("Enter something: ");
34     input = Console.ReadLine();
35     if (input == null) break;
36     if (!input.Equals("exit", StringComparison.OrdinalIgnoreCase))
37         Console.WriteLine($"You typed: {input}");
38 } while (!input.Trim().Equals("exit", StringComparison.OrdinalIgnoreCase));
39 Console.WriteLine("Exited input loop.");
40
41 Console.WriteLine("\nType a number to get its factorial, or 'exit' to quit.");
42 // string? input;
43 do
44 {
45     Console.Write("Input: ");
46     input = Console.ReadLine();
47     if (input is null) continue;
48     if (input.Trim().Equals("exit", StringComparison.OrdinalIgnoreCase)) break;
49     if (int.TryParse(input, out int n))
50     {
51         try
52         {
53             Console.WriteLine($"{n}! = {MathUtils.Factorial(n)}");
54         }
55         catch (Exception ex)
56         {
57             Console.WriteLine($"Error: {ex.Message}");
58         }
59     }
60     else
61     {
62         Console.WriteLine("Please enter a valid integer or 'exit'.");
63     }
64 } while (true);
65
66 Console.WriteLine("Exited input loop.");
67
68
69

```

Output

Show output from: Build

Build started at 19:00...

1>----- Build started: Project: stt-lab9, Configuration: Debug Any CPU -----

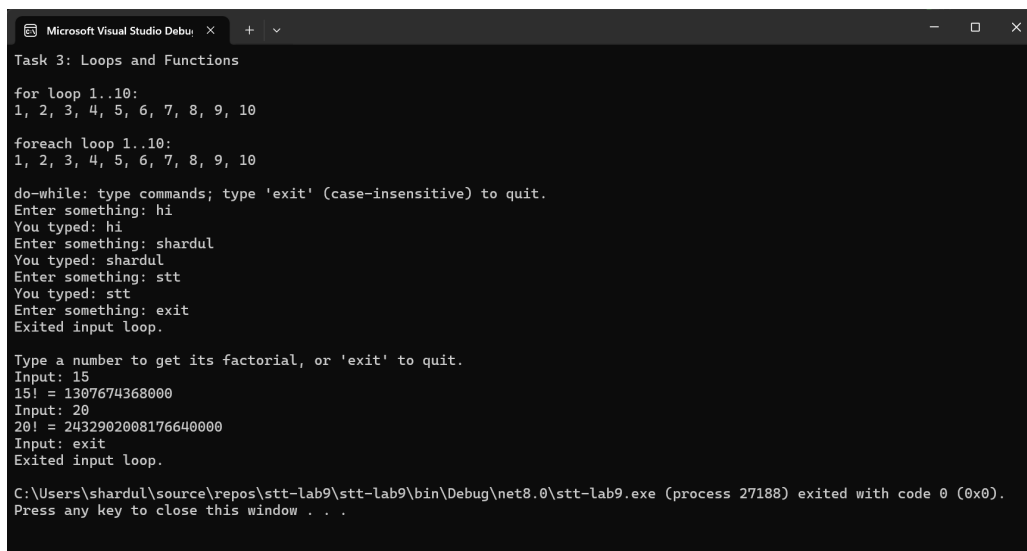
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.

1>stt-lab9 -> C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.dll

===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====

===== Build completed at 19:00 and took 00.298 seconds =====

Figure 12: Task3 code snippet



```

Task 3: Loops and Functions

for loop 1..10:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10

foreach loop 1..10:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10

do-while: type commands; type 'exit' (case-insensitive) to quit.
Enter something: hi
You typed: hi
Enter something: shardul
You typed: shardul
Enter something: stt
You typed: stt
Enter something: exit
Exited input loop.

Type a number to get its factorial, or 'exit' to quit.
Input: 15
15! = 1307674368000
Input: 20
20! = 2432902008176640000
Input: exit
Exited input loop.

C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 27188) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 13: Task3 interactive session output

1.4.4 Task 4 – Arrays and matrices

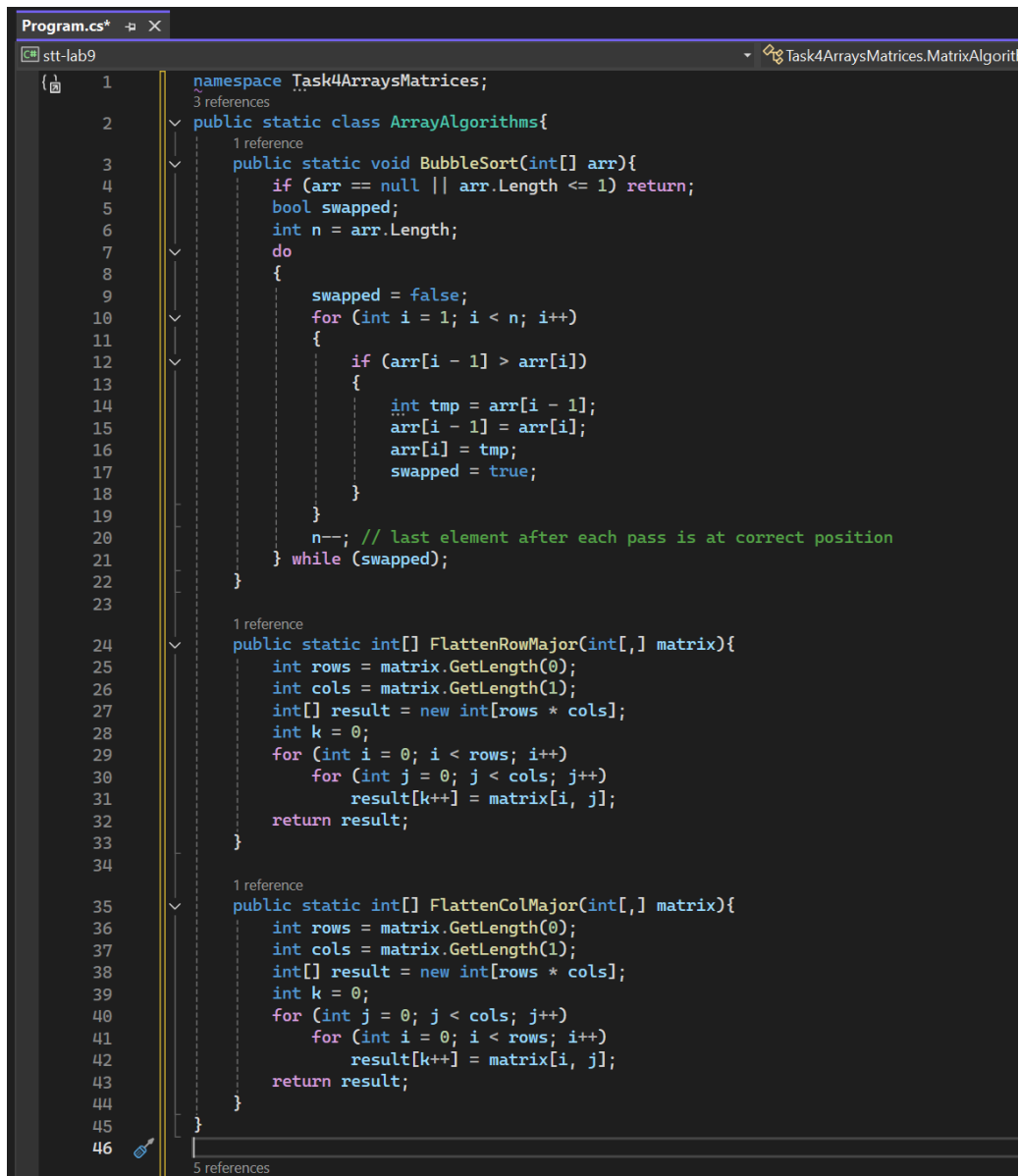
Design: Manual bubble sort with shrinking pass; row-major and column-major flattening; matrix multiply with dimension check and aligned printer.

Path: lab9/src/Task4ArraysMatrices/Program.cs

Run:

```
dotnet run --project lab9/src/Task4ArraysMatrices/Task4ArraysMatrices.csproj
```

Result: Sorted list echoed back; flattening shows differing orders; multiplication prints a 2x2 matrix from 2x3 and 3x2 inputs.



```
Program.cs*  stt-lab9  Task4ArraysMatrices.MatrixAlgorithms

1  namespace Task4ArraysMatrices;
2  public static class ArrayAlgorithms{
3      public static void BubbleSort(int[] arr){
4          if (arr == null || arr.Length <= 1) return;
5          bool swapped;
6          int n = arr.Length;
7          do
8          {
9              swapped = false;
10             for (int i = 1; i < n; i++)
11             {
12                 if (arr[i - 1] > arr[i])
13                 {
14                     int tmp = arr[i - 1];
15                     arr[i - 1] = arr[i];
16                     arr[i] = tmp;
17                     swapped = true;
18                 }
19             }
20             n--; // last element after each pass is at correct position
21         } while (swapped);
22     }
23
24     public static int[] FlattenRowMajor(int[,] matrix){
25         int rows = matrix.GetLength(0);
26         int cols = matrix.GetLength(1);
27         int[] result = new int[rows * cols];
28         int k = 0;
29         for (int i = 0; i < rows; i++)
30             for (int j = 0; j < cols; j++)
31                 result[k++] = matrix[i, j];
32         return result;
33     }
34
35     public static int[] FlattenColMajor(int[,] matrix){
36         int rows = matrix.GetLength(0);
37         int cols = matrix.GetLength(1);
38         int[] result = new int[rows * cols];
39         int k = 0;
40         for (int j = 0; j < cols; j++)
41             for (int i = 0; i < rows; i++)
42                 result[k++] = matrix[i, j];
43         return result;
44     }
45 }
46
```

Figure 14: Task4 Array Algorithms snippet

```

47 5 references
    public static class MatrixAlgorithms{
48      1 reference
49      public static int[,] Multiply(int[,] A, int[,] B)
50      {
51          int rA = A.GetLength(0), cA = A.GetLength(1);
52          int rB = B.GetLength(0), cB = B.GetLength(1);
53          if (cA != rB) throw new ArgumentException("Incompatible dimensions for multiplication.");
54          int[,] C = new int[rA, cB];
55          for (int i = 0; i < rA; i++)
56              for (int j = 0; j < cB; j++)
57              {
58                  int sum = 0;
59                  for (int k = 0; k < cA; k++)
60                      sum += A[i, k] * B[k, j];
61                  C[i, j] = sum;
62              }
63          return C;
64      }
65
66      4 references
67      public static void Print(int[,] M)
68      {
69          int rows = M.GetLength(0), cols = M.GetLength(1);
70          for (int i = 0; i < rows; i++)
71          {
72              for (int j = 0; j < cols; j++)
73              {
74                  Console.Write(M[i, j].ToString().PadLeft(6));
75              }
76              Console.WriteLine();
77          }
78      }

```

Figure 15: Task4 Matrix Algorithms snippet

```

79 0 references
80  public class Program
81  {
82      0 references
83      public static void Main(string[] args)
84      {
85          Console.WriteLine("Task 4: Arrays and Matrices\n");
86
87          // Bubble Sort
88          Console.WriteLine("Enter integers for bubble sort (space-separated):");
89          int[] arr = ReadIntArray();
90          ArrayAlgorithms.BubbleSort(arr);
91          Console.WriteLine("Sorted: " + string.Join(" ", arr));
92
93          // 2D -> 1D (Row-major / Column-major)
94          Console.WriteLine("\nDemo 2D flattening using sample matrix [[1,2,3],[4,5,6]]:");
95          int[,] sample = new int[,] { { 1, 2, 3 }, { 4, 5, 6 } };
96          Console.WriteLine("Sample Matrix:");
97          MatrixAlgorithms.Print(sample);
98          int[] rowMajor = ArrayAlgorithms.FlattenRowMajor(sample);
99          int[] colMajor = ArrayAlgorithms.FlattenColMajor(sample);
100          Console.WriteLine("Row-major: " + string.Join(" ", rowMajor));
101          Console.WriteLine("Column-major: " + string.Join(" ", colMajor));
102
103          // Matrix Multiplication
104          Console.WriteLine("\nMatrix Multiplication Demo (using small sample matrices)");
105          int[,] A = new int[,] { { 1, 2, 3 }, { 4, 5, 6 } }; // 2x3
106          int[,] B = new int[,] { { 7, 8 }, { 9, 10 }, { 11, 12 } }; // 3x2
107          int[,] C = MatrixAlgorithms.Multiply(A, B); // 2x2
108          Console.WriteLine("Matrix A:");
109          MatrixAlgorithms.Print(A);
110          Console.WriteLine("Matrix B:");
111          MatrixAlgorithms.Print(B);
112          Console.WriteLine("A x B = ");
113          MatrixAlgorithms.Print(C);

```

Figure 16: Task4 Program snippet

```

Task 4: Arrays and Matrices

Enter integers for bubble sort (space-separated):
> 10 9 8 7 6 5 4 3 1 1
Sorted: 1 1 3 4 5 5 6 7 8 9 10

Demo 2D flattening using sample matrix [[1,2,3],[4,5,6]]:
Sample Matrix:
  1  2  3
  4  5  6
Row-major:    1 2 3 4 5 6
Column-major: 1 4 2 5 3 6

Matrix Multiplication Demo (using small sample matrices)
Matrix A:
  1  2  3
  4  5  6
Matrix B:
  7  8
  9 10
 11 12
A x B =
  58  64
 139 154

C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 14632) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 17: Task4 interactive session output

Aggregate Run Commands

```

dotnet run --project lab9/src/Task1HelloWorld/Task1HelloWorld.csproj
dotnet run --project lab9/src/Task2Arithmetic/Task2Arithmetic.csproj
dotnet run --project lab9/src/Task3LoopsFunctions/Task3LoopsFunctions.csproj
dotnet run --project lab9/src/Task4ArraysMatrices/Task4ArraysMatrices.csproj

```

1.5 Output Reasoning (Tasks 5–7)

1.5.1 Task 5 – Level 0 (Code A)

Explanation: I ran the program containing a single post-increment expression inside `Console.WriteLine`. The post-increment operator returned the current value of `a` before performing the increment, so the console displayed 0. Immediately after evaluation, `a` became 1, but that new value was not part of the printed output. This confirmed the standard right-to-left evaluation of the post-increment without any hidden side effects.

Output I observed:

0

```

Program.cs
1 using System; //namespace
2 class Program //default visibility4 is 'internal' if not specified
3 {
4     public static void Main(string[] args)
5     {
6         int a = 0; //default visibility is 'private' (in a class)
7         Console.WriteLine(a++);
8     }
9 }

Output
Show output from: Build
Build started at 19:09...
1>----- Build started: Project: stt-lab9, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab9 -> C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 19:09 and took 00.289 seconds =====

```

Figure 18: L0A – code

```

0
C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 34328) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 19: L0A – output

1.5.2 Task 5 – Level 0 (Code B)

Explanation: I attempted to build the second variant where `Main` was defined as an instance method (`public void Main`). The C# compiler emitted error CS5001 because the runtime entry point must be a static method with a valid signature (e.g., `static void Main(string[] args)`). Since that requirement was not met, the assembly did not produce any executable output and the program never started.

Compiler behavior I observed:

error CS5001: Program does not contain a static 'Main' method suitable for an entry point

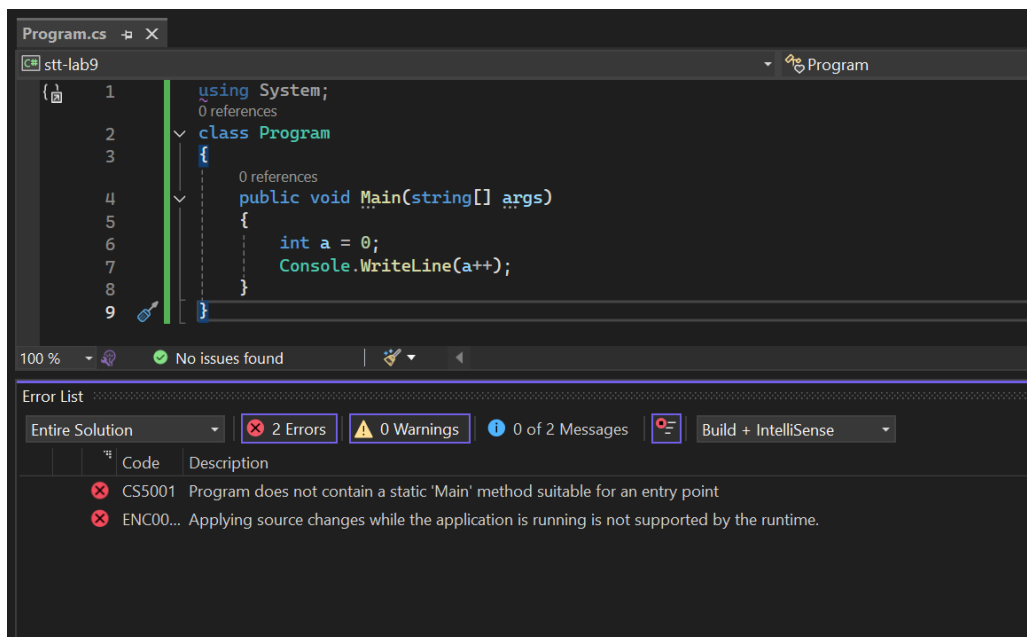


Figure 20: L0B – build error

1.5.3 Task 6 – Level 1 (Code A)

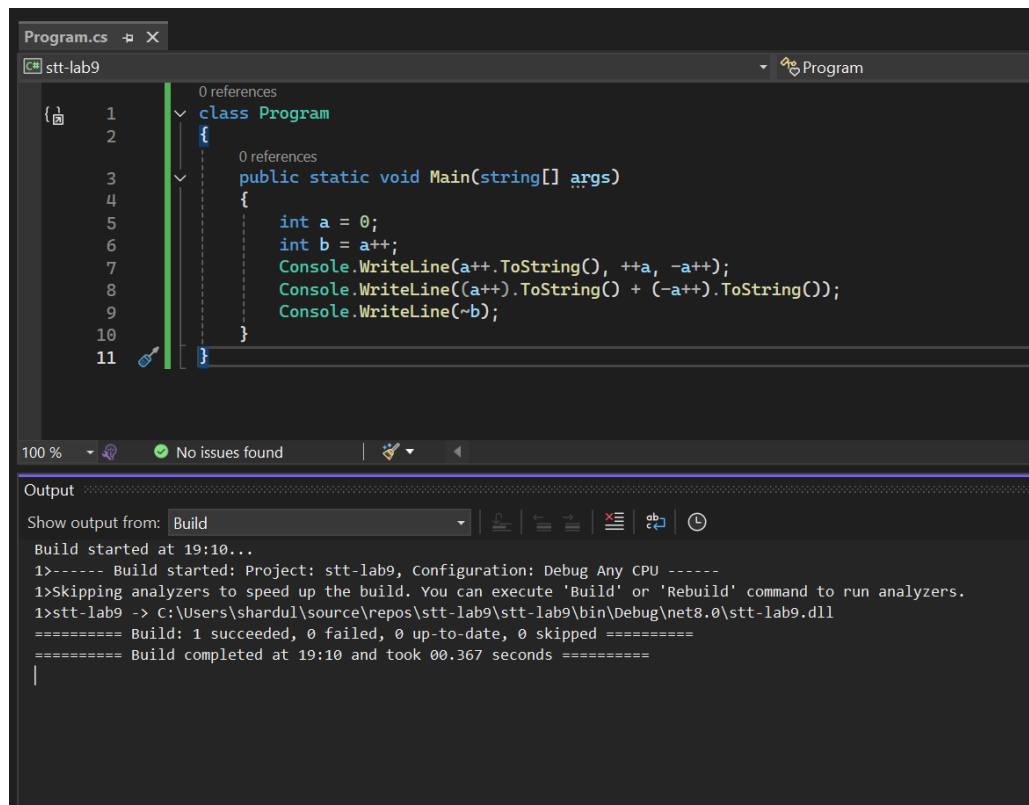
Explanation: I evaluated the sequence of increments carefully. First `b = a++` captured 0 then advanced `a` to 1. In `Console.WriteLine(a++.ToString(), ++a, -a++)`, the first argument became the format string "1" (after `a++` advanced `a` to 2), so the remaining evaluated arguments (`++a` making `a = 3`, and `-a++` yielding -3 then `a = 4`) were ignored for printing but still executed. The second line concatenated the string versions of 4 and -5 (with `a` moving from 4 to 6), yielding 4-5. Finally, `b` computed the bitwise complement of 0, producing -1.

Output I observed:

```

1
4-5
-1

```



```

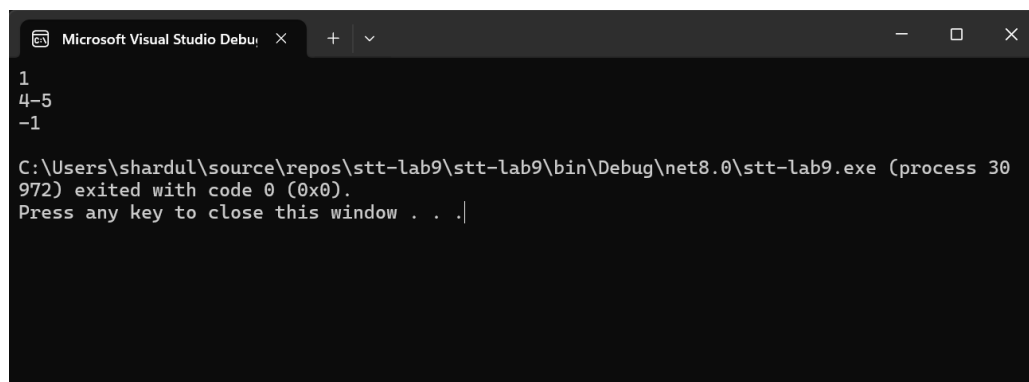
Program.cs
stt-lab9
0 references
class Program
{
    0 references
    public static void Main(string[] args)
    {
        int a = 0;
        int b = a++;
        Console.WriteLine(a++.ToString(), ++a, -a++);
        Console.WriteLine((a++).ToString() + (-a++).ToString());
        Console.WriteLine(~b);
    }
}

100% No issues found

Output
Show output from: Build
Build started at 19:10...
1>----- Build started: Project: stt-lab9, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab9 -> C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 19:10 and took 00.367 seconds =====

```

Figure 21: L1A – code



```

Microsoft Visual Studio Debug Console
1
4-5
-1

C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 30972) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 22: L1A – output

1.5.4 Task 6 – Level 1 (Code B: Top-level statements)

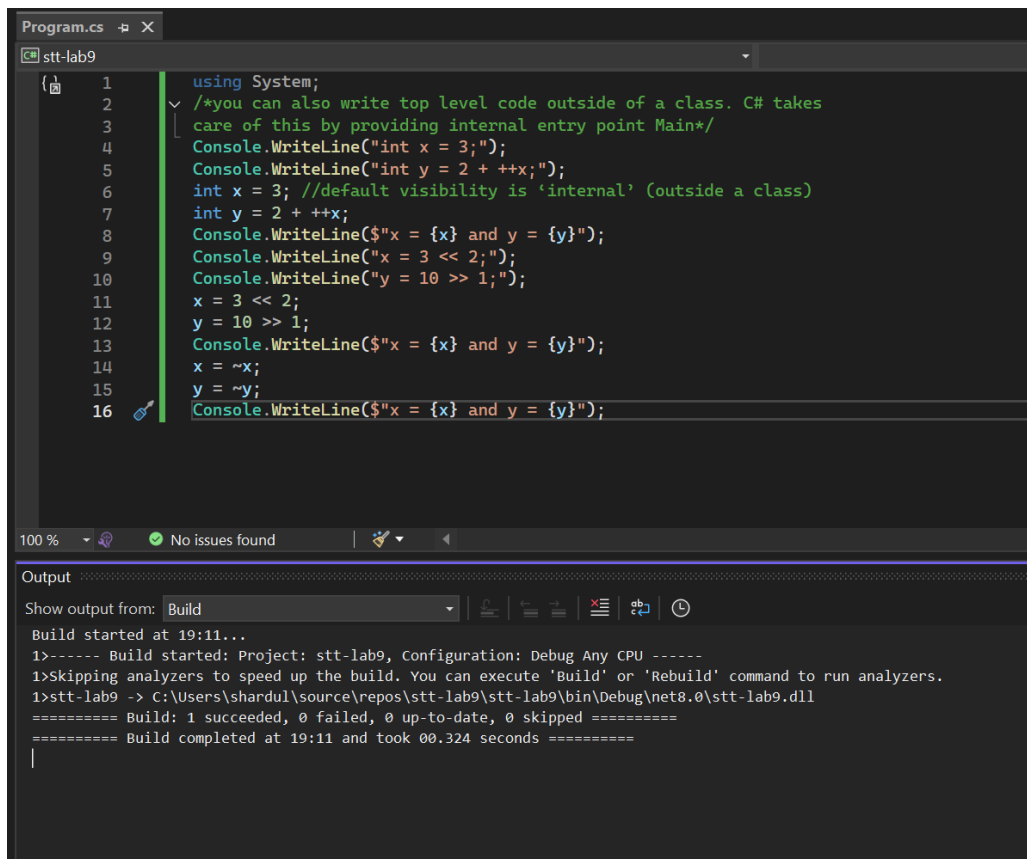
Explanation: I used top-level statements so the compiler generated an implicit `Main`. The pre-increment `++x` raised `x` from 3 to 4 before the addition, making `y = 2 + 4 = 6`. Left shift `3 << 2` multiplied the value by 4 yielding 12; right shift `10 >> 1` divided by 2 yielding 5. The bitwise complement operator applied two's-complement inversion, so 12 became -13 and 5 became -6.

Output I observed:

```

int x = 3;
int y = 2 + ++x;
x = 4 and y = 6
x = 3 << 2;
y = 10 >> 1;
x = 12 and y = 5
x = -13 and y = -6

```



```

Program.cs
stt-lab9
1  using System;
2  /*you can also write top level code outside of a class. C# takes
3   care of this by providing internal entry point Main*/
4  Console.WriteLine("int x = 3;");
5  Console.WriteLine("int y = 2 + ++x;");
6  int x = 3; //default visibility is 'internal' (outside a class)
7  int y = 2 + ++x;
8  Console.WriteLine($"x = {x} and y = {y}");
9  Console.WriteLine("x = 3 << 2;");
10 Console.WriteLine("y = 10 >> 1;");
11 x = 3 << 2;
12 y = 10 >> 1;
13 Console.WriteLine($"x = {x} and y = {y}");
14 x = ~x;
15 y = ~y;
16 Console.WriteLine($"x = {x} and y = {y}");

```

100 % No issues found

Output

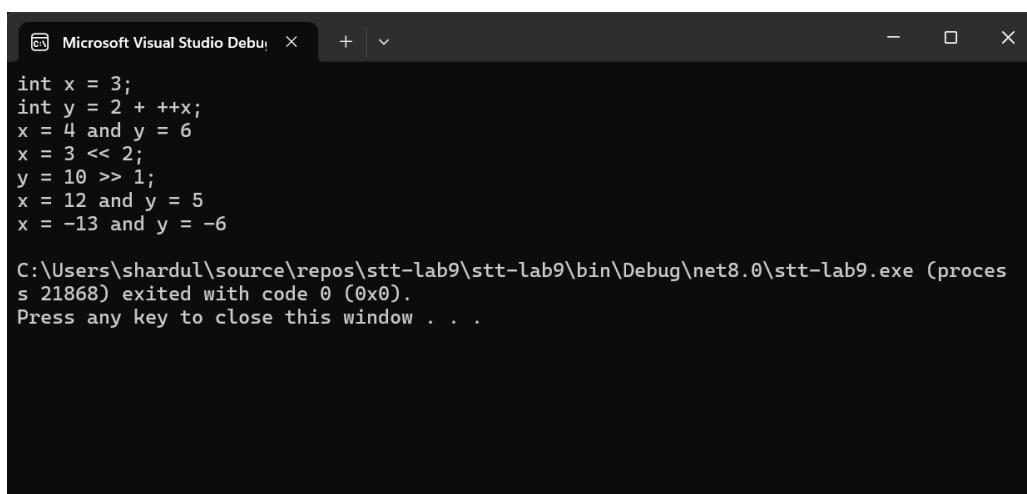
Show output from: Build

```

Build started at 19:11...
1>----- Build started: Project: stt-lab9, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab9 -> C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 19:11 and took 00.324 seconds =====

```

Figure 23: L1B – code



```

int x = 3;
int y = 2 + ++x;
x = 4 and y = 6
x = 3 << 2;
y = 10 >> 1;
x = 12 and y = 5
x = -13 and y = -6

C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 21868) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 24: L1B – output

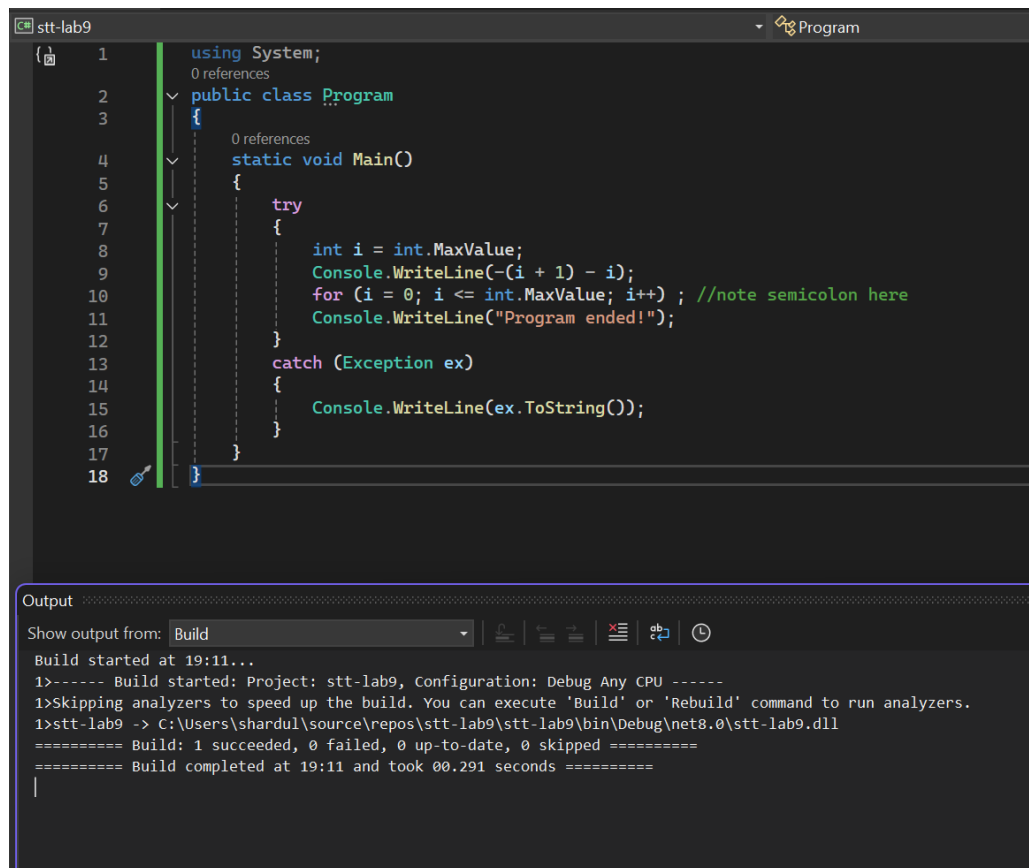
1.5.5 Task 7 – Level 2 (Code A)

Explanation: I initialized `i` to `int.MaxValue` and evaluated `-(i+1)-i` under the default unchecked arithmetic. The expression `i + 1` overflowed to `int.MinValue`; negating `int.MinValue` left it unchanged due to range asymmetry, and subtracting `i` yielded the wrapped result 1. The `for` loop's trailing semicolon created an empty loop body that repeatedly overflowed `i` and never allowed execution to reach the "Program ended!" line.

Output I observed:

1

(The program did not print "Program ended!" and did not terminate in a reasonable time.)



```

1  using System;
2  public class Program
3  {
4      static void Main()
5      {
6          try
7          {
8              int i = int.MaxValue;
9              Console.WriteLine(-(i + 1) - i);
10             for (i = 0; i <= int.MaxValue; i++) ; //note semicolon here
11             Console.WriteLine("Program ended!");
12         }
13         catch (Exception ex)
14         {
15             Console.WriteLine(ex.ToString());
16         }
17     }
18 }

```

Output

Show output from: Build

```

Build started at 19:11...
1>----- Build started: Project: stt-lab9, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab9 -> C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 19:11 and took 00.291 seconds =====

```

Figure 25: L2A – code

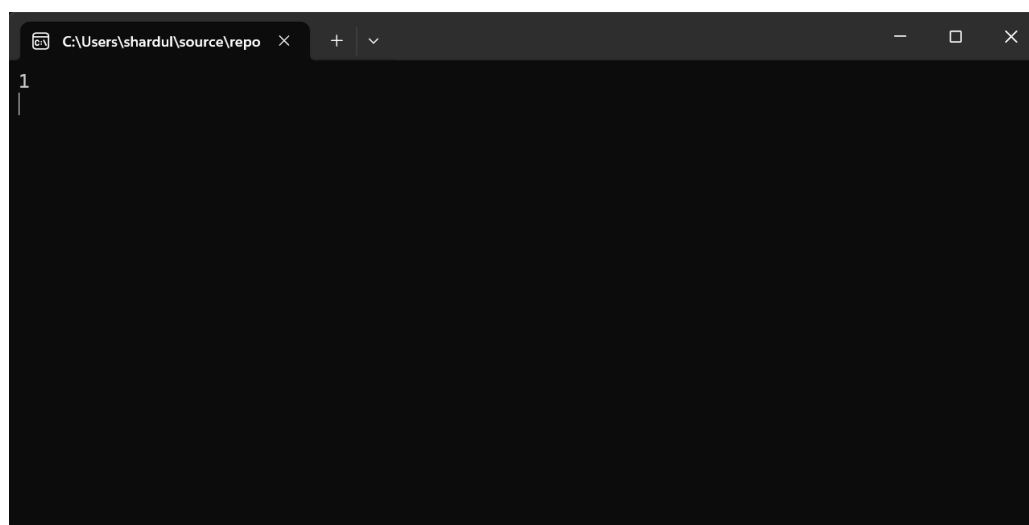


Figure 26: L2A – output

1.5.6 Task 7 – Level 2 (Code B)

Explanation: I ran the recursive program where `Main(["CS202"])` invoked itself without a termination condition. Each invocation consumed another stack frame until the process exhausted stack space. The console printed a long stack trace repeating the same frame many times, which is why it appeared to run for a while; after dumping the trace, the runtime terminated the process due to a stack overflow.

Behavior I observed:

```

...
...
...
at Program.Main(System.String[])
at Program.Main(System.String[])

```



```

at Program.Main(System.String[])
at Program.Main(System.String[])
at Program.Main(System.String[])
...
C:\Users\shardul\source\repos\stt-lab9\stt-lab9\bin\Debug\net8.0\stt-lab9.exe (process 24320) exited wi

```

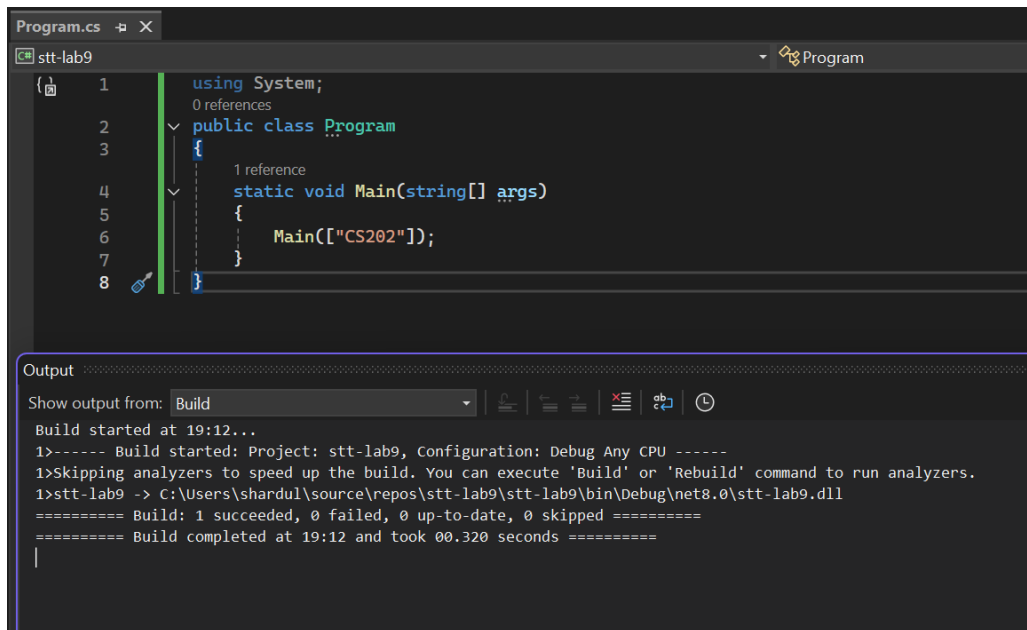


Figure 27: L2B – code

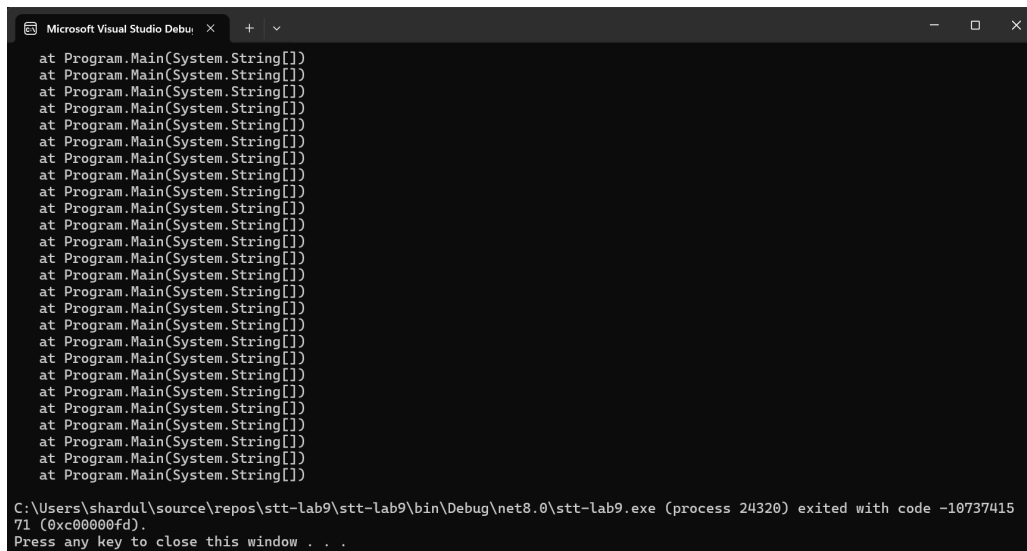


Figure 28: L2B – runtime termination

1.6 Conclusion

In this lab I implemented four focused C# console projects and verified core language features: input/output, arithmetic with simple classes, looping constructs, factorial computation, and array/matrix algorithms including bubble sort and multiplication. I also analyzed seven reasoning snippets, confirming operator semantics, overflow behavior, format string handling, and recursion limits. The work strengthened my practical understanding of evaluation order and memory effects while keeping the implementations minimal and explicit.

1.7 References

- Lab handout: lab9/TASK.md
- Microsoft C# docs: <https://learn.microsoft.com/en-us/dotnet/csharp>

- Arrays: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/arrays>
- Access modifiers: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers>
- Top-level statements: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/program-structure/top-level-statements>
- Exceptions and overflow: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/exception-handling>
- Collection expressions: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/collection-expressions>

2 Lab 10: Object Lifecycle and Polymorphism in C#

2.1 Introduction

This lab focuses on object lifecycle management and class relationships in C#. The first part deals with constructors, destructors, and tracking active objects. The second part explores inheritance and runtime polymorphism using virtual and overridden methods.

2.2 Setup and Environment

All programs in this lab were implemented and executed on a Windows environment using the .NET 8 runtime and Visual Studio IDE. The development setup ensured that C# features such as constructors, destructors, inheritance, and runtime polymorphism were properly compiled and observed during execution.

Software and System Configuration

- **Operating System:** Windows 10 / 11 (64-bit)
- **IDE Used:** Visual Studio 2022 (Community Edition)
- **Programming Language:** C# (Object-Oriented Programming)
- **Target Framework / Runtime:** Microsoft.NETCore.App 8.0.21
- **Compiler:** Roslyn C# Compiler (default in .NET SDK)

Project Configuration

- Project Type: Console Application
- Build Configuration: Debug (for destructor visibility during execution)
- Garbage Collection: Default .NET automatic GC (with manual invocation using `GC.Collect()` for demonstration)

Pre-Lab Preparation

Before beginning the implementation, the following prerequisites were ensured:

- Visual Studio was installed with the ‘.NET Desktop Development’ workload enabled.
- The .NET 8 SDK was properly configured in system environment variables.
- Basic understanding of class structure, fields, access modifiers, constructors, and destructors in C#.

This setup provided a stable environment for observing object lifecycle behavior (constructor invocation and garbage-collected destructor invocation), as well as supporting inheritance and dynamic method dispatch (virtual and override mechanisms).

2.3 Constructors and Data Control

2.3.1 Objective

The objective of this part of the lab was to understand how objects are created and destroyed in C#, how constructors and destructors function, and how a static variable can be used to keep track of the number of active objects in memory. Additionally, this part demonstrates assigning and displaying data stored inside objects.

2.3.2 Class Design

We define a class `Program` with:

- A private integer field `data`.
- A constructor that prints "Constructor Called" and increments a static counter to show how many objects are currently alive.

- A destructor that prints "Object Destroyed" and decrements the same static counter.
- A method `set_data(int x)` to assign a value to `data`.
- A method `show_data()` to display the stored value.

Code Implementation

```
class Program
{
    private int data;
    private static int liveCount = 0;

    public Program()
    {
        liveCount++;
        Console.WriteLine("Constructor Called");
        Console.WriteLine($"Active objects: {liveCount}");
    }

    ~Program()
    {
        liveCount--;
        Console.WriteLine("Object Destroyed");
        Console.WriteLine($"Active objects: {liveCount}");
    }

    public void set_data(int x)
    {
        data = x;
    }

    public void show_data()
    {
        Console.WriteLine($"data = {data}");
    }
}
```

2.3.3 Creating and Using Objects

In the `Main()` method, we dynamically created three objects, assigned values, and printed them. To ensure destructors run visibly, garbage collection was manually triggered after the objects went out of scope.

```
static void CreateAndUseProgramObjects()
{
    Program p1 = new Program();
    Program p2 = new Program();
    Program p3 = new Program();

    p1.set_data(10);
    p2.set_data(20);
    p3.set_data(30);

    p1.show_data();
    p2.show_data();
    p3.show_data();
}

// Call the method and force garbage collection in Main()
CreateAndUseProgramObjects();
GC.Collect();
GC.WaitForPendingFinalizers();
```

2.3.4 Observed Output

```
-- Constructors, Data Control and Destructors Demo --
Constructor Called
Active objects: 1
Constructor Called
Active objects: 2
Constructor Called
Active objects: 3
data = 10
data = 20
data = 30
Object Destroyed
Active objects: 2
Object Destroyed
Active objects: 1
Object Destroyed
Active objects: 0
```

2.3.5 Explanation of Output

- Each time a new object is created, the constructor runs and increases the `liveCount`. This confirms that the constructor is automatically invoked when an object instance is allocated.
- Each time an object is destroyed (after garbage collection), the destructor runs and decrements the counter.
- The destructor does **not** run immediately when the object goes out of scope. In C#, destructors are called by the garbage collector, which works asynchronously. Therefore, we forced destruction using:

```
GC.Collect();    GC.WaitForPendingFinalizers();
```

- The output clearly shows the object lifecycle: creation → usage → destruction.

2.4 Inheritance and Method Overriding

2.4.1 Objective

The objective of this part of the lab was to demonstrate how inheritance allows classes to reuse and extend functionality from base classes, and how method overriding enables derived classes to modify behavior. This part also illustrates runtime polymorphism using a base-class reference to invoke overridden methods.

2.4.2 Base Class: Vehicle

We begin by defining a base class `Vehicle` that contains shared properties and functionality for all vehicles.

- Protected fields `speed` and `fuel` allow derived classes to directly access these values.
- A virtual method `Drive()` reduces fuel by 5 and prints a general driving message.
- A virtual method `ShowInfo()` displays the current speed and fuel.

Code Implementation

```
public class Vehicle
{
    protected int speed;
    protected int fuel;

    public Vehicle(int speed = 0, int fuel = 0)
    {
        this.speed = speed;
        this.fuel = fuel;
    }

    public virtual void Drive()
```

```
{
    fuel -= 5;
    Console.WriteLine("Vehicle is moving...");
}

public virtual void ShowInfo()
{
    Console.WriteLine($"Vehicle Info - Speed: {speed}, Fuel: {fuel}");
}
}
```

2.4.3 Derived Class: Car

The class `Car` extends `Vehicle` and introduces a new field `passengers`. It overrides:

- `Drive()` to reduce fuel by 10 and print a message specific to cars.
- `ShowInfo()` to also display number of passengers.

Code Implementation

```
public class Car : Vehicle
{
    public int passengers;

    public Car(int speed = 0, int fuel = 0, int passengers = 0) : base(speed, fuel)
    {
        this.passengers = passengers;
    }

    public override void Drive()
    {
        fuel -= 10;
        Console.WriteLine("Car is moving with passenger");
    }

    public override void ShowInfo()
    {
        Console.WriteLine($"Car Info - Speed: {speed}, Fuel: {fuel}, Passengers: {
            passengers}");
    }
}
```

2.4.4 Derived Class: Truck

Similarly, the `Truck` class adds a new field `cargoWeight`, and overrides both methods to reflect heavier fuel usage and additional cargo data.

Code Implementation

```
public class Truck : Vehicle
{
    public int cargoWeight;

    public Truck(int speed = 0, int fuel = 0, int cargoWeight = 0) : base(speed, fuel)
    {
        this.cargoWeight = cargoWeight;
    }

    public override void Drive()
    {
        fuel -= 15;
```

```
        Console.WriteLine("Truck is moving with cargo");
    }

    public override void ShowInfo()
    {
        Console.WriteLine($"Truck Info - Speed: {speed}, Fuel: {fuel}, CargoWeight: {
            cargoWeight}");
    }
}
```

2.4.5 Demonstration of Runtime Polymorphism

To observe overriding behavior, we store one object of each class in a `Vehicle[]` array and call `Drive()` and `ShowInfo()` through the base-class reference.

```
Vehicle[] vehicles = new Vehicle[3];
vehicles[0] = new Vehicle(50, 100);
vehicles[1] = new Car(80, 60, 4);
vehicles[2] = new Truck(40, 200, 1500);

foreach (var veh in vehicles)
{
    veh.Drive();
    veh.ShowInfo();
    Console.WriteLine();
}
```

2.4.6 Observed Output

Vehicle is moving...

Vehicle Info - Speed: 50, Fuel: 95

Car is moving with passenger

Car Info - Speed: 80, Fuel: 50, Passengers: 4

Truck is moving with cargo

Truck Info - Speed: 40, Fuel: 185, CargoWeight: 1500

2.4.7 Explanation of Output

- Although the array type is `Vehicle[]`, the actual methods executed are determined by the **runtime type** of each object.
- This behavior is known as **runtime polymorphism**.
- The `virtual` keyword in the base class and the `override` keyword in derived classes enable dynamic dispatch.
- Because of this, `veh.Drive()` calls the appropriate version depending on whether `veh` refers to a `Vehicle`, `Car`, or `Truck`.

2.5 Output Reasoning (Level 0)

Expression Snippet 1

```
int a = 3;
int b = a++;
Console.WriteLine($"a is {a++}, b is {--b}");
int c = 3;
int d = ++c;
Console.WriteLine($"c is {c--}, d is {~d}");
```

Output:

a is 4, b is -4
 c is -4, d is -5
 Final values -> a=5, b=4, c=3, d=4

Reasoning:

- `a++` returns the old value first, then increments.
- `++b` increments first, then returns the value.
- The unary `+` and `-` before variables apply sign.
- Bitwise complement `d` flips all bits, which changes positive to negative value by: $x = -x - 1$.

Expression Snippet 2

[16] [0] [1] [1]

Reasoning:

- `"010%".Replace('0','%')` becomes `"%%1%"` which has length 4.
- Left shift: $4 \ll 2 = 16$.
- `"010%".Split('1')[1][0]` selects the `'0'` after `'1'`.
- `"010%".Split('0')[1][0]` selects the `'1'` after the first `'0'`.
- `"123".ToCharArray()[~-1]` uses `~-1` (bitwise NOT of 1, i.e., -2) as the index, giving `'1'`.

Expression Snippet 3

1, 3, 12, 0, 0

Reasoning:

- The loop moves all non-zero numbers to the front while maintaining their order.
- Zero elements are moved to the end.
- Final arrangement becomes: non-zero values first (1, 3, 12), followed by zeros.

2.6 Output Reasoning (Level 1)

Expression Snippet 1 — Strings, Indexing, and Bit Shifts

Observed output: [16] [0] [1] [1]

Reasoning:

1. `"010%".Replace('0','%') ⇒ "%%1%",` so `k = 4`.
2. `new Program().age` starts at 0; the constructor executes `age = (age==0 ? age+1 : age-1)`, so `age = 1`; then `++new Program().age` makes it 2.
3. Left shift: $4 \ll 2 = 16 ⇒$ first bracket [16].
4. `"010%".Split('1')[1][0]`: splitting on `'1'` gives `["0", "0%"]`, index [1][0] is `'0' ⇒ [0]`.
5. `"010%".Split('0')[1][0]`: splitting on `'0'` gives `["", "1", "%"]`, index [1][0] is `'1' ⇒ [1]`.
6. `"123".ToCharArray()[~-1]`: `~-1` is bitwise NOT of -1 , i.e., 0; index 0 is `'1'`; converting/ parsing yields `1 ⇒ [1]`.

Expression Snippet 2 — For-loop Tricks and Side Effects

Observed output:

```
run 1
2
4
6
8
10
11

run 2
11

run 3
0
```

Reasoning (key lines):

1. Construction: `new int()` is 0; `"0".Length` is 1; hence `new Program(0+1)` sets `p.f = 1`.
2. The for-header has an *empty body* (note the semicolon):
`for (int i=0, _=i; i<5 && ++p.f >=0; i++, Console.WriteLine(p.f++));`
 Each iteration does:
 - Pre-increment `++p.f` (used in the condition), then
 - Post-increment print `p.f++`.

Step-by-step with initial `p.f=1`: prints 2, 4, 6, 8, 10 and ends with `p.f=11`.

3. Next block:

```
for (; p.f == 0;); // condition false (11), so skip
{
    Console.WriteLine(p.f); // prints 11
}
```

Hence the trailing 11 after “run 1”.

4. **run 2:** `p = new Program(p.f) ⇒ p.f = 11`, prints 11.
5. **run 3:** default ctor `Program()` $\Rightarrow$ `f = 0`, prints 0.

Expression Snippet 3 — Events, Delegate Removal, and Null-conditional Calls

Observed output:

```
f1
f2
0
1
2
3
4
5
-6
-5
-4
-3
-2
-1
0
1
2
```

3
4
5

Reasoning:

1. Two handlers are added: A.f1 then B.f1; invoking p.handler() prints f1 then f2 in subscription order.
2. The code tries to remove handlers using new A/B instances:
p.handler -= new B().f1; p.handler -= new A().f1;. In .NET, instance-method delegates are equal only if *both* the target instance and method match. Using fresh instances means removal fails—handlers remain subscribed.
3. destroy() recursively prints i then increments until i == 6. With static i = 0, p?.destroy() prints 0 1 2 3 4 5.
4. After p = null; i = -6; p?.destroy(); does nothing because p is null.
5. (new Program())?.destroy(); runs with the shared static i = -6, so it prints from -6 up to 5.

2.7 Output Reasoning (Level 2)

Expression Snippet 1 — Inheritance, Constructor Chaining, Field Hiding, Bitwise Complement

Observed output:

61214135
8
-8

Relevant structure (paraphrased):

```
class Institute { internal int i = 7; Institute(){ Console.Write("1"); }
    public virtual void info(){ Console.Write("2"); } }
class IITGN : Institute { public int i = 8;
    IITGN(){ Console.Write("3"); }
    IITGN(int _){ Console.Write("4"); }
    public override void info(){ Console.Write("5"); } }
Main(){
    Console.Write("6");
    Institute ins1 = new Institute(); ins1.info();
    IITGN ab101 = new IITGN(3);
    ab101 = new IITGN(); ab101.info(); Console.WriteLine();
    Console.WriteLine(ab101.i);
    Console.WriteLine(~(((Institute)ab101).i));
}
```

Why it prints 61214135:

1. Console.Write("6") ⇒ prints 6.
2. new Institute() runs its ctor ⇒ prints 1.
3. ins1.info() uses dynamic dispatch, but object is Institute ⇒ prints 2.
4. new IITGN(3): ctor chaining first calls base Institute() (1), then IITGN(int) (4) ⇒ prints 14.
5. new IITGN(): base Institute() (1), then IITGN() (3) ⇒ prints 13.
6. ab101.info() is overridden ⇒ prints 5.

Concatenating: 6 1 2 1 4 1 3 5 → 61214135.

Why the lines 8 and -8:

- ab101.i accesses the *derived* field IITGN.i (field hiding), value 8 ⇒ prints 8.
- Casting (Institute)ab101 switches field binding to base Institute.i (value 7). Bitwise complement rule: $\sim x = -x - 1$, so $\sim 7 = -8$ ⇒ prints -8.

Expression Snippet 2 — Multicast Delegates: Order and Removal

Observed output:

```
run 1
fun1()
fun2()
run 2
fun2()
fun1()
run 3
fun1()
```

Relevant structure (paraphrased):

```
public delegate void mydel();
mydel obj1 = new mydel(p.fun1); obj1 += new mydel(p.fun2);
mydel obj2 = new mydel(p.fun2); obj2 += new mydel(p.fun1);
obj2 -= p.fun2;
```

Why it prints this way:

1. **Run 1:** Multicast invocation preserves subscription order. `obj1` has `fun1` then `fun2` $\Rightarrow$ prints `fun1()` then `fun2()`.
2. **Run 2:** `obj2` subscribed `fun2` then `fun1` $\Rightarrow$ prints `fun2()` then `fun1()`.
3. **Run 3:** `obj2 -= p.fun2`; removes the *matching* invocation (same target instance and method), leaving only `fun1` $\Rightarrow$ prints `fun1()`.

Note: For instance methods, delegate removal succeeds only if both the target object and method match the one in the invocation list.

Expression Snippet 3 — ArrayList, Aliasing, and Side Effects

Observed output:

```
1
1
202
202
1
111
```

Relevant structure (paraphrased):

```
ArrayList L = new ArrayList();
L.Add(new Program()); L.Add(new Program()); // Program has int x;
for (i=0;i<L.Count;i++) Console.WriteLine(++((Program)L[i]).x);

L[0] = L[1];
((Program)L[0]).x = 202;

for (i=0;i<L.Count;i++) Console.WriteLine(((Program)L[i]).x);

((Program)L[0]).x = 111;
L.RemoveAt(0);
Console.WriteLine(L.Count);
Console.WriteLine(((Program)L[0]).x);
```

Step-by-step reasoning:

1. Two distinct `Program` instances with default `x=0`.
2. First loop uses pre-increment: prints 1 then 1; now objects are $(x_0=1, x_1=1)$.
3. `L[0] = L[1]`; makes both entries reference the *same* second object (aliasing).

4. Setting `((Program)L[0]).x = 202`; updates the shared object. Next loop prints 202, 202.
5. Then `((Program)L[0]).x = 111`; updates the same shared object to 111.
6. `L.RemoveAt(0)`; removes the first reference; list now has one element (the former second), so `L.Count` is 1.
7. The remaining element is the shared object with `x=111`, so the last line prints 111.

Key idea: `ArrayList` stores *references*; assigning `L[0] = L[1]` does not clone objects, it aliases them.

2.8 Output Reasoning (Level 3)

Expression Snippet 1 — Constructor chaining, overriding, field hiding, and bitwise complement

Observed output:

```
61214135
8
-8
```

Relevant code sketch:

```
public class Institute {
    internal int i = 7;
    public Institute() { Console.Write("1"); }
    public virtual void info() { Console.Write("2"); }
}
public class IITGN : Institute {
    public int i = 8; // hides Institute.i
    public IITGN() { Console.Write("3"); }
    public IITGN(int _) { Console.Write("4"); }
    public override void info() { Console.Write("5"); }
}
static void Main() {
    Console.Write("6");
    Institute ins1 = new Institute(); ins1.info();
    IITGN ab101 = new IITGN(3); // base() is implicit
    ab101 = new IITGN(); // base() is implicit
    ab101.info();
    Console.WriteLine();
    Console.WriteLine(ab101.i);
    Console.WriteLine(~(((Institute)ab101).i));
}
```

Why this prints exactly that:

1. `Console.Write("6")` $\Rightarrow$ prints 6.
2. `new Institute()` calls *its* ctor $\Rightarrow$ prints 1.
3. `ins1.info()` calls `Institute.info` (no override on `ins1`) $\Rightarrow$ prints 2.
4. `new IITGN(3)`: in C#, a derived ctor implicitly calls `base()` first, then its own body. Hence prints 1 (base) then 4 (derived) $\Rightarrow$ 14.
5. `new IITGN()` similarly prints 1 (base) then 3 (derived) $\Rightarrow$ 13.
6. `ab101.info()` dispatches virtually to the override in `IITGN` $\Rightarrow$ prints 5.
7. Putting the pieces together (no spaces): 6 1 2 1 4 1 3 5 $\Rightarrow$ 61214135.
8. `ab101.i` accesses `IITGN`'s public field (hides the base field) $\Rightarrow$ prints 8.
9. `((Institute)ab101).i` accesses the *base* field (`internal int i = 7`). Bitwise complement obeys $\sim x = -x - 1$, so $\sim 7 = -8 \Rightarrow$ prints -8.

Expression Snippet 2 — Multicast delegates: combination, order, and removal

Observed output:

```
run 1
fun1()
fun2()
run 2
fun2()
fun1()
run 3
fun1()
```

Relevant code sketch:

```
public delegate void mydel();
public void fun1() { Console.WriteLine("fun1()"); }
public void fun2() { Console.WriteLine("fun2()"); }
static void Main() {
    Program p = new Program();
    mydel obj1 = new mydel(p.fun1);
    obj1 += new mydel(p.fun2);
    Console.WriteLine("run 1"); obj1();

    mydel obj2 = new mydel(p.fun2);
    obj2 += new mydel(p.fun1);
    Console.WriteLine("run 2"); obj2();

    obj2 -= p.fun2;
    Console.WriteLine("run 3"); obj2();
}
```

Why this prints exactly that:

1. Delegate invocation preserves *subscription order*.
 - obj1 list: [fun1, fun2] $\Rightarrow$ prints fun1() then fun2().
 - obj2 list: [fun2, fun1] $\Rightarrow$ prints fun2() then fun1().
2. obj2 -= p.fun2; removes the first matching occurrence of fun2 from obj2's invocation list, leaving [fun1]. Hence "run 3" prints only fun1().

Expression Snippet 3 — Reference aliasing with ArrayList and field mutation

Observed output:

```
1
1
202
202
1
111
```

Relevant code sketch:

```
using System.Collections;
class Program { public int x;
    public static void Main() {
        ArrayList L = new ArrayList();
        L.Add(new Program()); // L[0]
        L.Add(new Program()); // L[1]
        for (int i=0; i<L.Count; i++) Console.WriteLine(++((Program)L[i]).x);

        L[0] = L[1]; // now both slots reference the same object
        ((Program)L[0]).x = 202;
    }
}
```

```
    for (int i=0; i<L.Count; i++) Console.WriteLine(((Program)L[i]).x);

    ((Program)L[0]).x = 111;
    L.RemoveAt(0);
    Console.WriteLine(L.Count);
    Console.WriteLine(((Program)L[0]).x);
}
}
```

Why this prints exactly that:

1. Initially, two distinct `Program` objects with `x = 0`. The prefix increment loop prints:
 - For `L[0]`: `++x` $\Rightarrow$ prints 1.
 - For `L[1]`: `++x` $\Rightarrow$ prints 1.
2. `L[0] = L[1]`; makes both `L[0]` and `L[1]` refer to the *same* second object (aliasing). Setting `((Program)L[0]).x = 202` therefore affects *both* slots. Printing both elements now yields: 202 then 202.
3. Next, mutate through the shared reference: `((Program)L[0]).x = 111;`. `L.RemoveAt(0);` drops the first slot; the remaining `L[0]` still refers to that same (only) object. Thus, `L.Count` prints 1 and `((Program)L[0]).x` prints 111.